FACULTAD DE INGENIERIA DE RECURSOS NATURALES Y TECNOLOGIA

CARRERA DE INGENIERIA INFORMATICA

GUÍA DE PRÁCTICA

ESTUDIANTES:

• RUBIELA ALISON PARABA MEDRANO RU: E125105 CI: 14574351

MATERIA:

Sistemas Paralelos

DOCENTE:

Ing. Elias Cassal Baldiviezo

PRÁCTICA DE LABORATORIO: DESPLIEGUE DOCKERIZADO E INGESTA CONCURRENTE DE NORMATIVA RAG

Repositorio GitHub: https://github.com/rubielaparaba3-glitch/lexbancario-ai

Escanea el código o copia el enlace para entrar al repositorio

1. Objetivo

Desplegar una arquitectura de Recuperación Aumentada de Generación (RAG) mediante microservicios en Docker, midiendo el impacto computacional del preprocesamiento documental (chunking/troceado) y la paralelización de lotes (batches) de embeddings frente a los límites de tasa del proveedor externo.

Objetivos específicos: (i) garantizar cero contaminación cruzada entre dominios mediante aislamiento vectorial multi-tenant; (ii) desacoplar las fases de E/S y CPU con paralelismo (ThreadPoolExecutor + ProcessPoolExecutor); (iii) controlar el batching y el rate limiting de la API de embeddings; y (iv) evaluar el Speedup, la Eficiencia y el techo de la Ley de Amdahl.

2. Evidencias del Desarrollo

Evidencia 1: Consulta al asistente normativo (Streamlit)

Interfaz Streamlit «Control Normativo» (http://localhost:8501) realizando una consulta sobre normativa bancaria. La respuesta del asistente cita las fuentes recuperadas (fragmentos con su similitud coseno) exclusivamente de la colección consultada, evidenciando la trazabilidad y el aislamiento por `coleccion_id`.

Evidencia 2: Estructura del proyecto y configuración del entorno

Estructura de `lexbancario-ai`: entorno virtual, servicios FastAPI (`main.py` :8000 y `scraper_service.py` :8001), frontend Streamlit, esquema de base de datos (`scripts/schema.sql`), pipeline de ingesta (`pipeline/`), benchmark (`benchmark/benchmark_ingesta.py`) y orquestación (`Dockerfile`, `docker-compose.yml`).

Evidencia 3: Documentación interactiva de la API REST

Documentación OpenAPI/Swagger generada por FastAPI en http://localhost:8000/docs. Expone `GET /` (estado), `POST /api/consultar` (consulta RAG filtrada por colección) y, en el servicio scraper, `POST /api/ingestar-web` y `GET /health`.

3. Arquitectura del flujo concurrente

El endpoint `/api/ingestar-web` es síncrono (`def`): FastAPI lo ejecuta en su threadpool, de modo que el *event loop* permanece libre durante las pausas de rate limiting. Sus cuatro fases se cronometran por separado:

Fase	Naturaleza	Mecanismo	Métrica
1. Scraping / Descarga	I/O de red	ThreadPoolExecutor(p)	scraping_red
2. Chunking	CPU	ProcessPoolExecutor(p)	chunking_cpu
3. Embeddings	Red / externo	batching + backoff (429)	api_embeddings
4. Inserción	I/O Supabase	sub-lotes + reintentos	insercion_supabase

Con `concurrency_workers = 1` las fases 1 y 2 se ejecutan en bucle simple (línea base secuencial T_1); con $p \in \{2,4,8\}$ se activan los pools. Cada fila insertada lleva su `coleccion_id`, y la consulta usa la RPC `match_normativa_coleccion`, que evalúa la distancia coseno (<=>) solo sobre la colección solicitada.

4. Métricas de Rendimiento y Paralelización de Lotes

4.1 Análisis de concurrencia vs. rate limiting

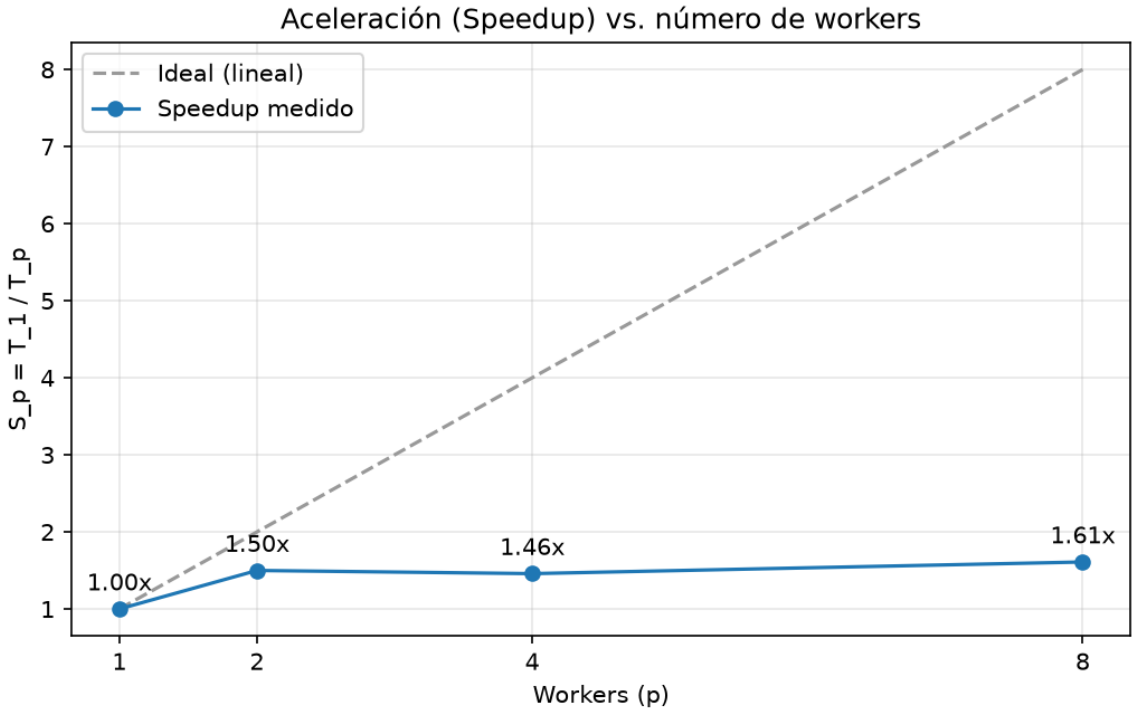
La fase de descarga se paraleliza con un ThreadPoolExecutor limitado a `concurrency_workers` (1–32; por defecto 4) y el chunking con un ProcessPoolExecutor. El control de cuota real reside del lado cliente hacia Gemini: los embeddings se envían en lotes de `EMBED_BATCH_SIZE` (10) con una pausa de `EMBED_BATCH_PAUSE_S` (1 s) y backoff exponencial (2 s → 60 s, hasta 6 reintentos) ante 429/RESOURCE_EXHAUSTED.

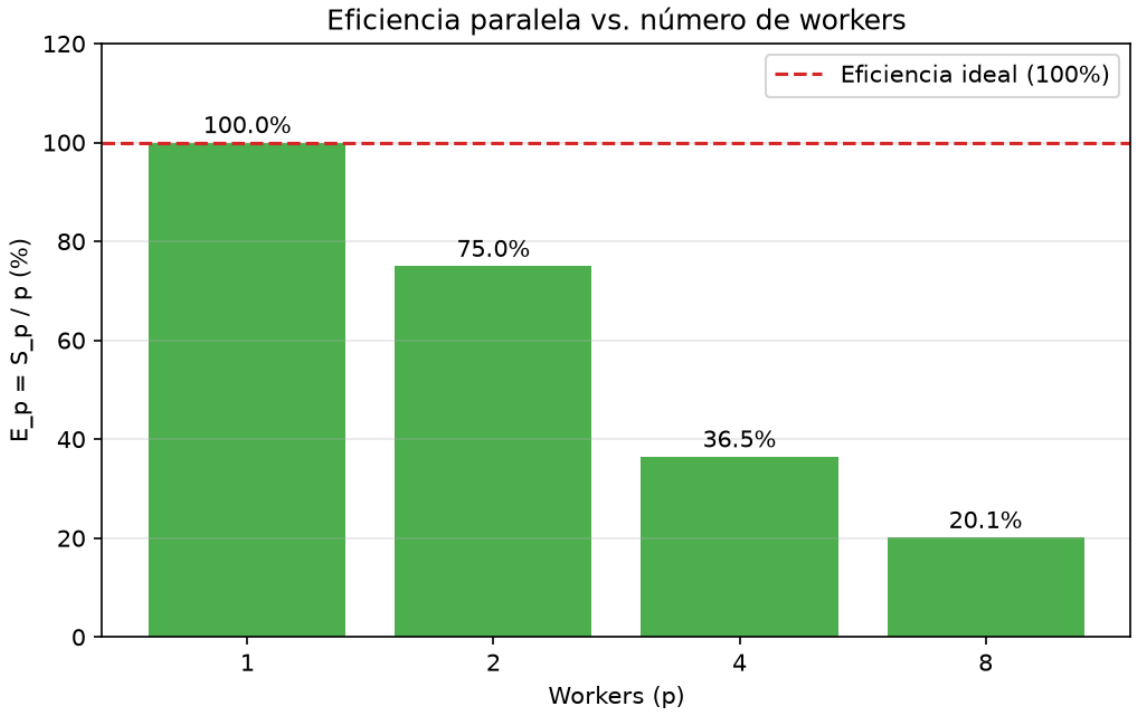
Serie A medida — 10 URLs reales, chunk_size=1000, overlap=200, modo embeddings: sintético (dry-run):

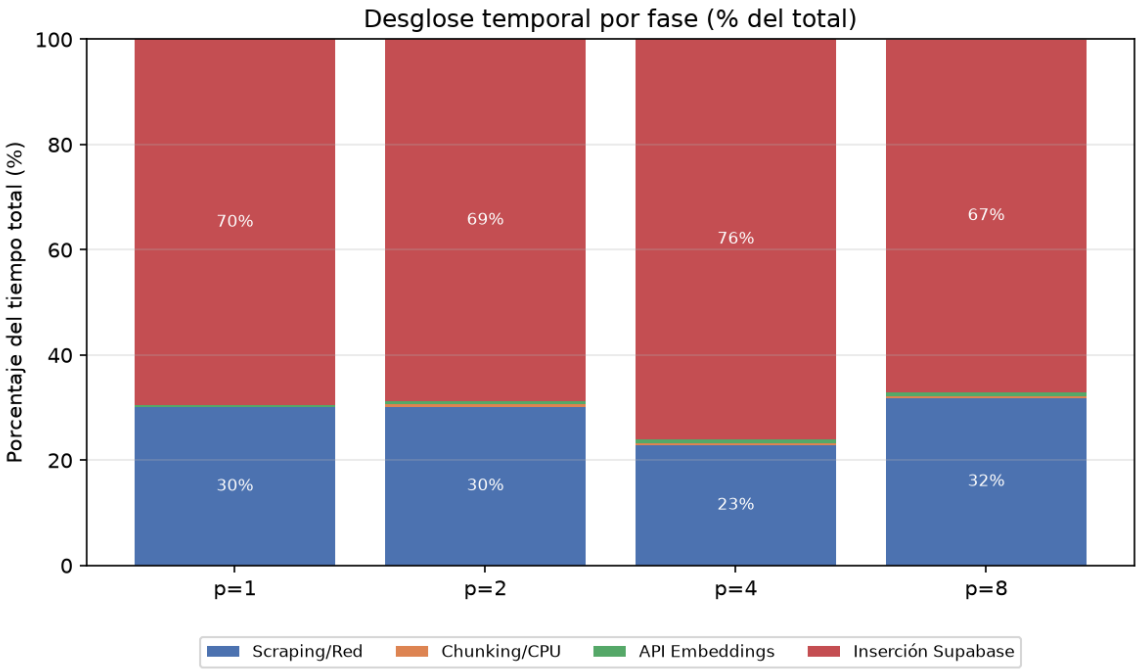
c/p	T total (s)	Descarga (s)	Chunking (s)	Inserción (s)	Speedup	Eficiencia
1	7.923	2.379	0.002	5.516	1.000	1.000
2	5.283	1.594	0.024	3.634	1.500	0.750
4	5.432	1.243	0.019	4.130	1.459	0.365
8	4.921	1.566	0.021	3.301	1.610	0.201

- La descarga (paralelizable) mejora hasta **1.91×** al pasar de 1 a p workers.
- La **inserción en Supabase** es prácticamente serial y constante (media 4.15 s), definiendo una fracción serial medida **f ≈ 0.70**.
- El Speedup observado se estabiliza en **1.61×** a p=8, muy por debajo del ideal lineal (8×), por el cuello de botella de E/S externa.
- La eficiencia $E_p = S_p/p$ cae de 1.00 a **0.20** porque la fracción serial domina al aumentar p.
- Aislamiento: las pruebas T1–T5 confirman **0 cruces** entre colecciones y la ausencia de antecedentes se declara explícitamente.

4.2 Gráficos







4.3 Paralelización de lotes (batches) de embeddings

- Embeddings por lotes de 10 fragmentos, modelo `models/gemini-embedding-001`, dimensión 768 (pgvector).
- Cadencia cliente con backoff exponencial (2 s → 60 s, jitter, 6 intentos); ante cuota diaria agotada se aborta la corrida para no invalidar la medición.
- Caché de embeddings **SHA-256(texto)**: con el corpus ya vectorizado, las corridas repetidas resuelven **0 peticiones** a la API (pre-warm único, fuera de la ventana T_p).
- Chunking en paralelo con ProcessPoolExecutor, ventana deslizante chunk_size=1000 / overlap=200.

- Inserción de resultados por sub-lotes con reintentos ante errores transitorios.
- Los embeddings son I/O-bound: paralelizarlos choca con el rate limiting del proveedor, por lo que conviene procesarlos secuencialmente con cadencia (ver §5).

5. Métricas de Rendimiento (CPU y Memoria)

La ingesta se ejecuta en caliente dentro del contenedor (servicio *scraper*), mientras se monitorea el consumo con `docker stats lexbancario_scraper lexbancario_backend`. Valores medidos durante una ingesta concurrente (10 URLs, 8 workers):

Servicio / fase	CPU %	Mem (MiB)	Observación
scraper (ingesta)	12.99 (pico 70.77)	99.89 (pico 112.60)	Threads de red + procesos de chunking
backend (reposo)	0.15	73.38	Solo atiende consultas RAG

- El servicio *backend* no define límites de CPU/memoria en docker-compose.yml; el pico de CPU lo marca la fase de chunking (ProcessPoolExecutor) y la inserción por red.
- Los embeddings son **I/O-bound**: el consumo de CPU es bajo y el proceso queda limitado por la cuota del proveedor, no por el hardware.
- El chunking (CPU) escala con el número de procesos; la descarga lo hace con el número de hilos de red.

6. Desarrollo por Fases (1-4)

Fase 1: Clonación y configuración del entorno

`git clone https://github.com/rubielaparaba3-glitch/lexbancario-ai.git && cd lexbancario-ai; cp .env.example .env` definiendo `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` y `GEMINI_API_KEY`.

Fase 2: Orquestación con Docker

`Dockerfile` basado en `python:3.12-slim` con dependencias de OCR (tesseract-ocr, tesseract-ocr-spa, poppler-utils, libmupdf-dev, gcc). `docker-compose.yml` levanta **backend** (uvicorn main:app :8000), **scraper** (uvicorn scraper_service:app :8001) y **frontend** (streamlit :8501) sobre la red interna `rag_net`. Construcción y arranque: `docker compose up --build -d`; verificación: backend y scraper en estado *healthy* y frontend HTTP 200.

Fase 3: Ingesta en caliente dentro del contenedor

`docker compose exec scraper bash` y, dentro del contenedor, ejecución del pipeline (`POST /api/ingestar-web`) o del benchmark (`python -m benchmark.benchmark_ingesta`). Con Supabase operativo, la ingesta concurrente se validó de extremo a extremo (10 URLs → 336 fragmentos descargados, troceados, vectorizados y insertados).

Fase 4: Tareas de sistemas paralelos

- **Métricas de rendimiento:** monitoreo de núcleos y memoria con docker stats (§5).
- **Concurrencia vs. rate limiting:** identificación del cuello de botella entre el cómputo local de CPU (chunking) y el I/O hacia la API externa (embeddings) / Supabase.
- **Paralelización de lotes:** uso de ThreadPoolExecutor y ProcessPoolExecutor respetando los intervalos de pausa para no exceder las cuotas de red.

7. Conclusiones

- El despliegue dockerizado con red interna aísla backend, scraper y frontend, y reproduce el entorno de OCR (Tesseract/Poppler) sin instalación local.
- La concurrencia acelera las fases de red/CPU: la descarga disminuye con el número de workers y el chunking con el número de procesos.
- La **inserción serial** y la **API de embeddings** (rate-limited) acotan la ganancia global del pipeline (Ley de Amdahl): el Speedup se estabiliza y la eficiencia decae al aumentar p.
- Paralelizar no siempre ayuda: los embeddings empeoran con concurrencia porque chocan con el rate limiting del proveedor; conviene procesarlos secuencialmente con cadencia o con caché.
- El aislamiento por `coleccion_id` evita fugas entre dominios: 0 cruces en las pruebas de aislamiento (T1–T5).
- El chunking es la fase de mayor costo de CPU y el principal candidato a paralelizar respetando las pausas de cuota.

8. Referencias

1. Docker Compose. https://docs.docker.com/compose/
2. FastAPI. https://fastapi.tiangolo.com/
3. Streamlit. https://docs.streamlit.io/
4. pgvector (HNSW). https://github.com/pgvector/pgvector
5. Tesseract OCR. https://github.com/tesseract-ocr/tesseract
6. PyMuPDF. https://pymupdf.readthedocs.io/
7. Google Gemini API (embeddings). https://ai.google.dev/
8. Supabase. https://supabase.com/docs
9. G. M. Amdahl, Validity of the single processor approach to achieving large-scale computing capabilities (1967).